

Advanced 3D Game Development

CA3 – Reflective document

Brona Keevers-Roux

20098883

Repo: <https://github.com/hillyleopard133/4th-year-game-dev/tree/ca3-3d>

Presentation: <https://youtu.be/2lGim59Xlwc>

Unity Video: https://youtu.be/w_kQTSFPpZs

Rendering Foundations

The rendering approach used in the current vertical slice builds on the physically based rendering workflow established in CA1, but is applied within a simpler and more controlled scene. The main change since CA1 is not the introduction of new rendering techniques, but a tightening of how lighting and materials interact under a reduced lighting setup.

The project continues to follow a standard PBR workflow where Base Colour textures are treated as sRGB data, while all data-driven textures such as Metallic, Roughness, Ambient Occlusion, and Normal maps are imported with sRGB disabled. This ensures that these maps are interpreted as linear data rather than colour information, which is essential for maintaining physically accurate material response.

In practice, this becomes more important in the current scene because lighting is significantly simplified compared to CA1. Without multiple fill or rim lights to compensate for material inaccuracies, any incorrect texture import settings or PBR misconfiguration becomes immediately visible. This has reinforced the importance of consistent import discipline rather than relying on lighting to “fix” material issues.

The lighting model itself is intentionally minimal. The scene uses a single directional light as the primary source of illumination, supported by a number of point lights placed around the environment. The directional light establishes overall scene direction, shadowing, and mood. It acts as the structural foundation of the lighting model.

Point lights are used in positions that correspond to practical ceiling fixtures within the environment, acting as motivated light sources rather than purely artistic additions. Their placement is therefore grounded in the physical layout of the scene, reinforcing the idea that they exist as part of the environment rather than being externally designed lighting helpers. These lights provide localised illumination where it would naturally occur in an industrial space, while maintaining a deliberately low intensity to preserve the intended abandoned factory atmosphere.

The overall lighting is intentionally subdued rather than fully illuminated, supporting the narrative of a partially derelict or inactive space. This means the point lights are not used to fully brighten areas for readability, but instead to suggest limited functional lighting within an otherwise darkened environment. Compared to CA1, where additional fill and rim lights were used to actively sculpt the subject, the current approach relies more on environmental plausibility and restraint, with contrast emerging from naturally occurring light placement rather than layered lighting design.

Post-processing is used conservatively to reinforce the atmosphere of the environment without overpowering the underlying lighting model. A Global Volume was configured using ACES tonemapping, subtle vignette, bloom, and colour adjustments. Bloom intensity was intentionally kept low (0.5) so that emissive and practical light sources produced a slight glow without overwhelming the darker industrial setting. Colour adjustments were used to slightly cool and desaturate the image, helping reinforce the abandoned factory atmosphere, while the vignette subtly darkens the frame edges to increase focus and tension within the scene. Despite these additions, the scene still relies primarily on physically based lighting and material response rather than aggressive post-processing to achieve readability and mood.

Unreal Engine 5 and Unity URP share the same physically based rendering principles, but differ in how they expose and manage them. Unreal provides a more standardised material workflow where Roughness, Metallic, and Normal inputs are consistently handled within a single PBR system, whereas Unity requires more manual configuration, particularly around Smoothness versus Roughness interpretation. For effects such as dissolve materials, Unreal typically relies on a Masked opacity workflow, while Unity allows more procedural construction through Shader Graph, offering greater flexibility at the cost of setup complexity. Lighting also differs in defaults: Unreal generally produces more physically consistent results out of the box, while Unity URP requires additional tuning of lights, shadows, and ambient settings to achieve equivalent visual fidelity.

Fig 1: Before post processing



Fig 2: After post processing



Optimisation and Constraints

The optimisation strategy for the project is based on maintaining stable real-time performance within a small vertical slice environment rather than attempting large-scale optimisation techniques. The scene is intentionally constrained in terms of lighting complexity, shader usage, and real-time effects.

The most significant optimisation decision is the use of a minimal lighting model. A single directional light provides global illumination and shadowing, while a number of point lights are used for localised illumination. This reduces overall rendering cost by avoiding large numbers of overlapping dynamic lights, which are expensive in a real-time pipeline such as Unity URP.

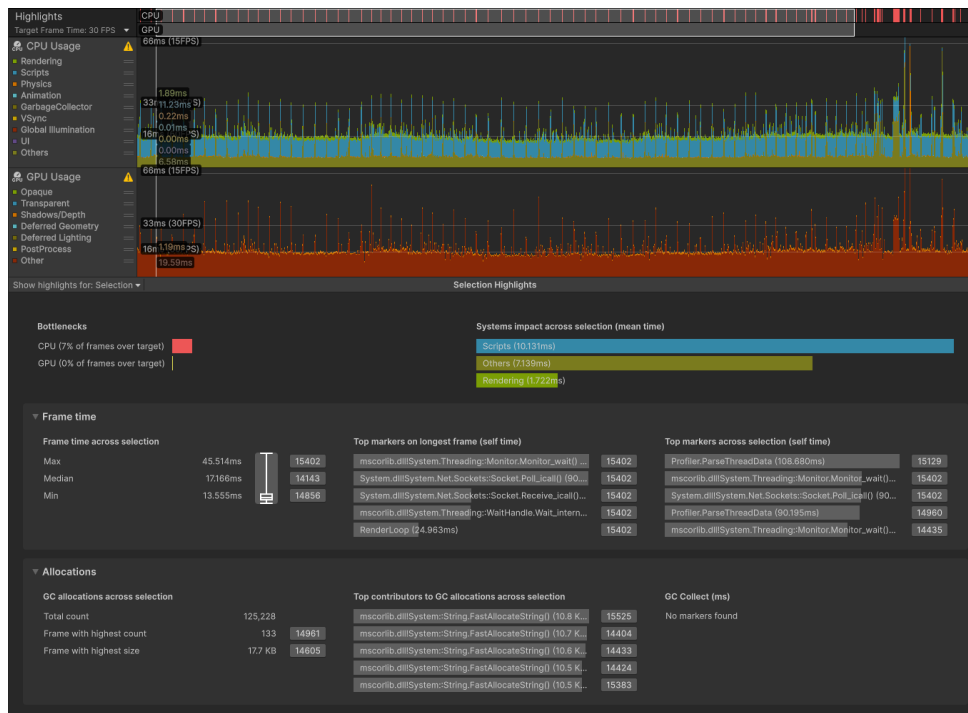
Shadow cost is also controlled through limiting shadow casting to the directional light. Point lights are either non-shadow casting or used in a constrained manner. This avoids the high performance cost associated with multiple real-time shadow maps, which scale poorly as light count increases.

The material system is deliberately kept lightweight. Standard URP Lit materials are used throughout the scene, with a single custom Shader Graph effect used for the dissolve/burn-out material. This shader is designed to remain computationally simple, relying on basic Smoothstep masking and noise rather than layered or multi-pass shading techniques. This ensures that shader complexity does not become a bottleneck.

Texture handling also contributes to performance stability. Correct import settings ensure that no unnecessary runtime conversions are required, and materials behave consistently under lighting without requiring compensatory shader logic.

Post-processing is minimal and not used as a core rendering dependency. This reduces full-screen rendering overhead and ensures that performance is driven primarily by geometry, lighting, and material cost rather than screen-space effects.

Fig 3 Profiling



Profiling identified periodic spikes within `NetworkRunner.Update`, which correspond to Fusion’s internal network tick. These spikes are expected, as Fusion batches input polling, simulation stepping, and state replication within a single update cycle. No sustained CPU bottleneck was observed, and frame times remained stable outside of network tick processing. This indicates that networking overhead is present but controlled within acceptable bounds for a small-scale shared session.

The main trade-off in this design is the reduction of advanced rendering features such as baked global illumination or complex lighting rigs in favour of simplicity and controllability. While these features could improve visual richness, they are not necessary for the scale of the current scene and would introduce unnecessary complexity.

Networking Implementation

The implemented networked feature is a minimal multiplayer system using Photon Fusion 2 in Shared Mode. The system allows multiple clients to join a session and spawn a single controllable player character per client. The focus of the implementation is validating core session behaviour rather than building full gameplay systems.

Each player is spawned using Fusion's `NetworkRunner.Spawn()` method, with the joining `PlayerRef` passed into the spawn call to assign `InputAuthority`. This ensures that each client has exclusive control over their own player object. `StateAuthority` is handled by Fusion's Shared Mode simulation model, where authority is distributed across connected peers depending on object ownership.

Early iterations of the system incorrectly assumed a server-authoritative structure. This led to the incorrect use of server checks during spawning logic, which prevented consistent player instantiation in Shared Mode. The issue was resolved by removing server-based assumptions and relying on Fusion's `OnPlayerJoined` callback, which is triggered consistently across all peers when a new player enters the session.

Once callback registration was correctly ordered before `StartGame`, player spawning became stable and deterministic. This highlighted that in Fusion, execution order and callback registration are more important than the complexity of the logic itself.

The system uses Fusion's networked simulation model rather than RPC-based communication for player state. This is appropriate because player movement represents continuous state rather than discrete events. RPCs are reserved for event-driven interactions, while transform and movement data are handled through networked simulation updates.

A limitation of the current implementation is that custom networked state is limited to a small number of variables, such as player health and name. While these [Networked] properties demonstrate correct use of Fusion's state replication model, the wider gameplay systems still rely heavily on default transform synchronisation and do not yet extend into more complex states such as interaction systems, combat events, or authoritative gameplay logic. This restricts extensibility, particularly for systems that would benefit from a hybrid approach combining continuous state replication with event-based RPC communication.

Authentication is handled using Unity Authentication Services. Players sign in anonymously before entering a session, and the resulting authentication state is used to validate identity locally. A server-side validation proxy was explored but not fully implemented due to scope constraints. As a result, authentication currently functions as a client-side identity gate rather than a fully server-verified system.

Fig 4: Authentication in photon

Custom Server Provider

Authentication URL

<https://ca3-auth.azurewebsites.net/api/auth>

■ Reject all clients if not available.

Fig 5: Authentication evidence in unity console

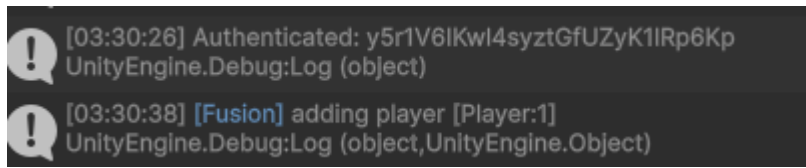


Fig 6: Two concurrent users

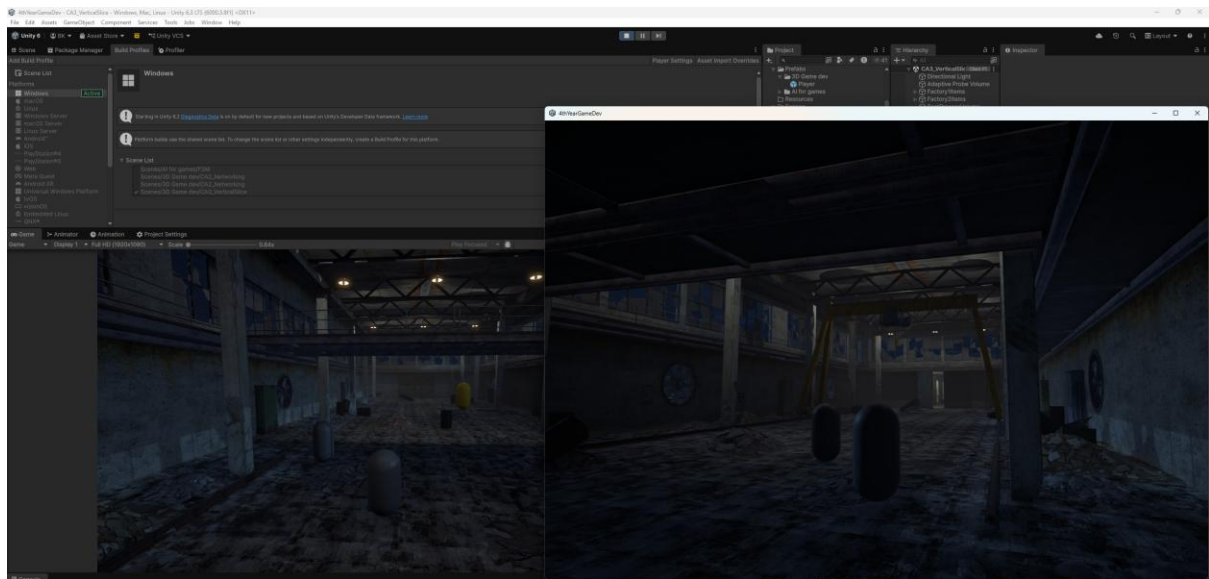


Fig 7: Photon Fusion CCU graph



Networking in Modern Engines

Photon Fusion 2 and Unreal Engine 5 both use authoritative networking, but they approach it in slightly different ways. Fusion exposes authority more directly through `InputAuthority` and `StateAuthority`, which allows simulation to be split across peers depending on the mode being used. In Shared Mode, this means there isn't a single strict server controlling everything, and clients can still own and simulate their own objects. Unreal Engine 5 is more rigid in comparison, relying on a traditional server-authoritative model where the server is always responsible for final game state and clients act as either owning or simulated proxies.

For replication, Fusion uses `[Networked]` properties, which are automatically synchronised as part of the network simulation. Unreal uses replicated `UPROPERTY` variables, which need to be explicitly registered through functions like `GetLifetimeReplicatedProps`. Unreal's approach takes more setup, but it makes it clearer what is being replicated and when.

For network events, Fusion uses RPCs without separating them into different types. Unreal splits RPCs into Server, Client, and NetMulticast versions, which makes it more obvious which side is running the code. This adds more structure, especially in larger projects, but also means more boilerplate.

The biggest difference between the two engines is flexibility versus structure. Fusion gives more freedom to build different kinds of multiplayer setups, including peer-based or shared simulation models. Unreal sticks to a stricter server-first approach, which tends to be more predictable and easier to reason about in competitive multiplayer games.

Most modern multiplayer systems, regardless of engine, rely on techniques like client prediction, interpolation, and server reconciliation to hide latency. These are standard across the industry and are widely used in real-time multiplayer games to keep movement and input feeling responsive.

References:

Epic Games (2024) *Unreal Engine Networking Overview*

<https://docs.unrealengine.com/5.3/en-US/networking-and-multiplayer-in-unreal-engine/>

Unity Technologies *Photon Fusion Documentation*

<https://doc.photonengine.com/fusion/current/getting-started/fusion-intro>

Glenn Fiedler (GDC) *Networking for Physics Programmers*

<https://www.gdcvault.com/play/1022195/Physics-for-Game-Programmers-Networking>

Version Control and Professional Workflow

Version control was managed using a feature-branch workflow across the project, with work separated into dedicated branches for major systems before being merged back into the main branch once stable. For this submission, a ca3-submit-3d tag was used to mark the final state of the project, providing a clear reference point for assessment.

This approach helped keep development for different systems (such as rendering, scene setup, and networking) isolated from each other, reducing the risk of unstable changes affecting the main build. It also made it easier to test features incrementally without disrupting the overall project structure.

Throughout development, commits were generally made during key milestones or completed features rather than at a strictly atomic level. While this was sufficient to track overall progress, it occasionally made it harder to isolate specific changes when debugging issues that involved multiple systems interacting at once.

There were also periods where work progressed faster than version control updates, particularly during experimentation phases. This reduced the granularity of the commit history, meaning some development steps are not fully captured in isolation.

Overall, the branching and tagging strategy provided a stable structure for managing the project at submission stage. In future development, a more consistent commit pattern with smaller, incremental changes would improve traceability and make debugging complex systems like networking easier to manage.

Fig 8: CA3 tags and branch

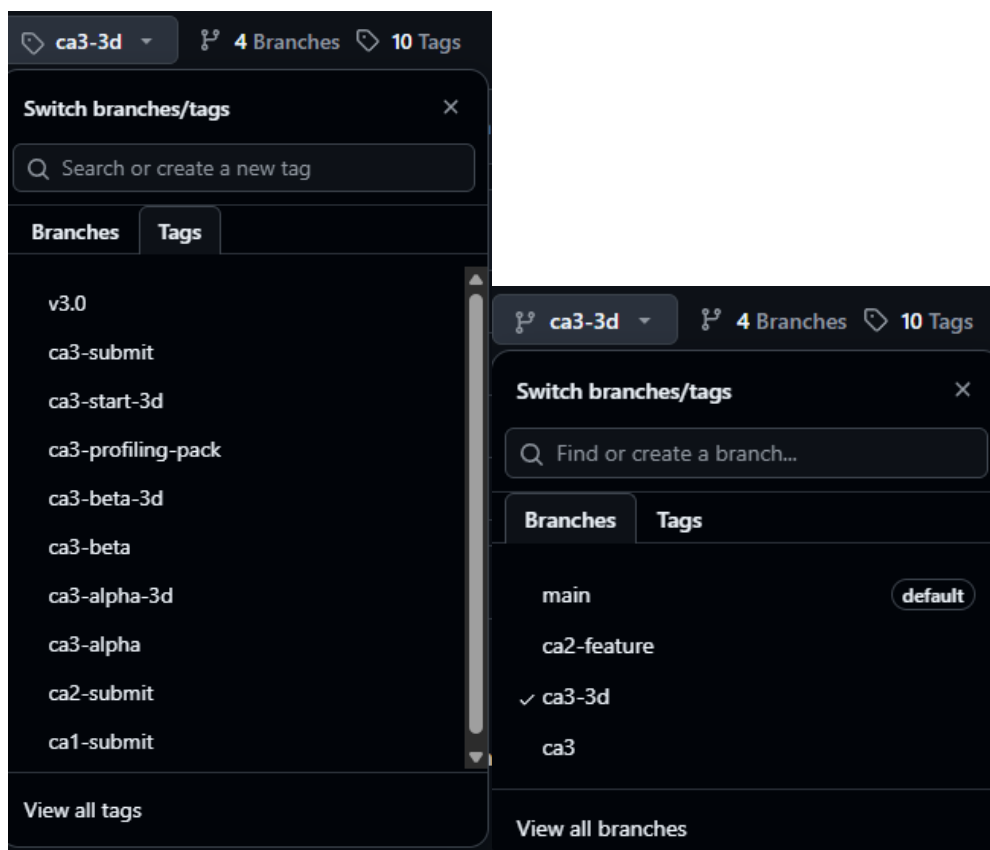
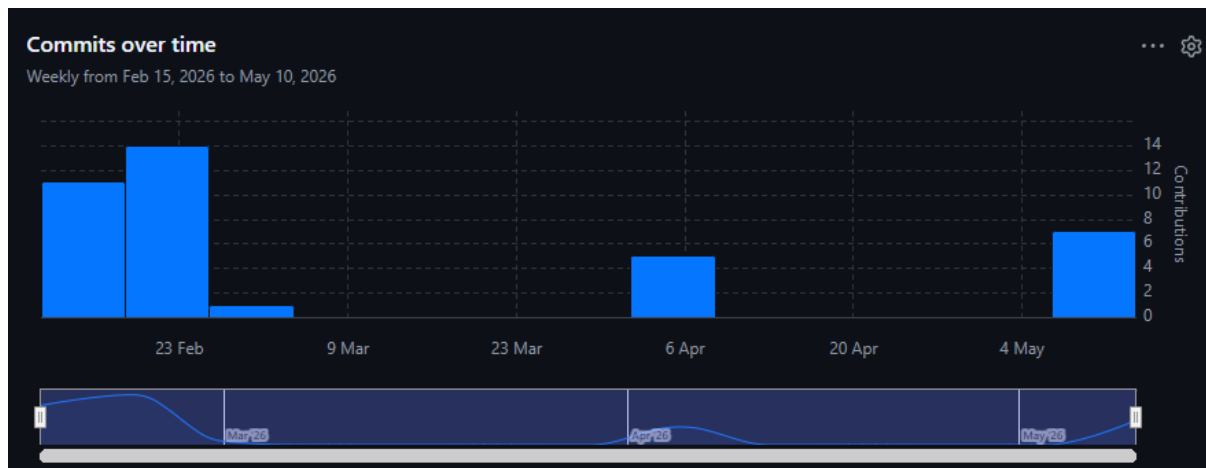


Fig 9: Commit graph



Overall Reflection

The most significant challenge across the module was developing a correct mental model of how Photon Fusion handles authority and simulation in Shared Mode. Early on, I approached it with a server-authoritative mindset, which led to incorrect assumptions about when and where state should be validated. This directly contributed to issues with player spawning and session behaviour, particularly when server-style checks were applied in a system that does not operate on a strict server-client hierarchy.

The main shift came when I understood that Fusion assigns authority at the object level rather than globally, and that Shared Mode distributes simulation responsibility across peers. Once this was clear, the behaviour of systems like `OnPlayerJoined` and `PlayerRef` became much more predictable. A lot of the earlier instability in spawning was ultimately caused by incorrect assumptions about execution order and authority context, rather than faults in the networking logic itself.

The decision I am most satisfied with is keeping the scope of the project deliberately small. Focusing on a minimal multiplayer loop—session creation, joining, and basic player spawning—meant I could isolate and verify core Fusion behaviour without introducing extra gameplay systems that would have made debugging significantly harder. Given how sensitive Fusion is to setup order and callback timing, this helped avoid situations where multiple overlapping systems were masking the actual cause of issues.

One of the more persistent problems during development was session instability during early testing. These issues were not related to gameplay code, but instead came from configuration and lifecycle ordering, such as when callbacks were registered relative to `StartGame`, and how the `NetworkSceneManager` was set up. This reinforced the idea that in multiplayer development, correctness is often determined more by engine integration order than by the logic inside scripts.

If I were to restart the project, I would separate the architecture more clearly from the beginning. In the current version, authentication, bootstrap logic, and networking setup are handled in closely related scripts, which made it harder to isolate where issues were coming from during debugging. Splitting these responsibilities more cleanly would make the system easier to reason about and reduce the risk of setup-related bugs.

Another improvement would be making more consistent use of explicit [Networked] variables earlier in development. While the current implementation does include basic networked state such as player health and identifiers, extending this pattern sooner would have made the relationship between gameplay state and replication clearer, particularly for future systems like interaction or combat logic.

Overall, the project reflects a functional but still developing understanding of multiplayer systems. The main takeaway is that stability in networked applications is less about complex gameplay code and more about correctly handling authority, lifecycle order, and system separation. Once those foundations are correct, building additional features becomes significantly more predictable.

Statement of Use of AI

AI assistance (ChatGPT) was used during the development of this submission as a support tool to speed up development and debugging.

It was used to clarify technical concepts related to Photon Fusion 2, particularly authority handling (InputAuthority vs StateAuthority), NetworkRunner lifecycle behaviour, and callback ordering issues affecting session stability and player spawning. It also assisted in diagnosing runtime issues such as incorrect setup order and Fusion configuration problems.

AI was also used during development to help write and refine sections of the implementation code, particularly around networking setup, authentication flow, and debugging Fusion session behaviour. This included iterating on solutions to ensure correct integration between Unity Authentication and Photon Fusion.

In addition, AI was used to speed up development by reducing time spent troubleshooting and by helping quickly test and refine different implementation approaches. This improved iteration speed when resolving networking issues and stabilising the project.